

Graph-Based versus Vector Retrieval-Augmented Generation for Multi-Hop Question Answering over Technical Documentation

Industry Collaboration with Everllence SE (formerly MAN Energy Solutions)

Student Name:	Necati Furkan Colak	Student ID:	24746323
Program:	MSc Computer Science	Semester:	Spring 2026
Supervisor:	[Supervisor Name]	Co-Supervisor:	[Everllence Contact]
Email:	necatifurkan.colak@uzh.ch	Date:	July 10, 2026

Abstract

Technical documents are organized: sections reference other sections, components appear in parts lists and in procedures, and a repair instruction points to a torque table three chapters away. Vector-based retrieval-augmented generation (RAG) ignores this structure. It retrieves passages by embedding similarity, which works for single-fact questions and degrades on questions whose evidence is spread over connected passages. Benchmark studies measure this directly: pipelines that answer single-fact questions well lose much of their accuracy on multi-hop queries [1]. Graph-based RAG builds a graph over documents (sections, entities, cross-references) and retrieves by traversing it; a peer-reviewed study in the manufacturing domain reports improved document question answering with this approach [2]. The two approaches have not been compared under controlled conditions on the same technical corpus with hop-stratified questions. This thesis builds both pipelines over one public corpus of technical documents, adds a hybrid variant, and measures answer correctness, retrieval recall, faithfulness, and indexing and query cost, stratified by the number of evidence hops. An ablation with a manually corrected graph quantifies how extraction errors affect final answers. The outcome states when building a document graph is worth its cost.

Motivation and Problem Statement

Operational context. Engineers ask questions whose answers require combining passages. “Which lubricant does the overhaul procedure require, and what is its storage temperature?” needs the procedure, a cross-referenced material sheet, and a storage table. Assistants built on vector RAG answer the first part and miss the rest, because no single passage matches the whole question. In maintenance and service settings, these multi-part questions are common, and a partial answer forces the engineer back into manual search.

Evidence for the gap. The MultiHop-RAG benchmark evaluated standard RAG pipelines on queries whose evidence spans several documents and found that they perform poorly on them, across retrieval variants [1]. Document GraphRAG, a peer-reviewed manufacturing-domain study, built a knowledge graph from document structure and keyword links and reports improved document question answering compared with vector retrieval [2]. Graph construction methods for text corpora are established [3]. What is missing is a controlled comparison: same corpus, same questions, same generator, hop-stratified analysis, and a cost account for graph construction. Published graph RAG studies also treat the extracted graph as given, so how extraction errors limit answer quality remains unmeasured.

Why cost matters. A graph must be extracted, stored, and maintained. If graph retrieval only helps on rare question types, organizations should know this before they invest. The comparison therefore reports quality and cost together, per question stratum.

Research Questions

1. **RQ1 (primary):** Does graph-based retrieval improve answer correctness over vector retrieval on multi-hop questions over technical documentation, and by how much per hop count?
2. **RQ2:** Does a hybrid retriever (graph traversal plus vector search) beat both single approaches, and on which question types?
3. **RQ3:** How do graph extraction errors (missing edges, wrong entities) propagate to answers, measured by comparing the automatically extracted graph with a manually corrected subset?
4. **RQ4:** What are the indexing and per-query costs of each approach?

Methodology

- **Literature review:** RAG foundations [4]; graph-based RAG [2, 3]; multi-hop RAG evaluation [1]; evaluation methods [5].
- **Data:** 100 to 300 public technical documents with internal cross-references (equipment manuals, public standards, maintenance handbooks). A QA set of 150 to 250 questions stratified by evidence hops (one, two, three or more), drafted with an LLM and fully verified by hand, with gold answers and the full evidence chain per question, split into development and held-out test sets.

- **Systems:** (a) Vector RAG: hybrid BM25 and dense retrieval with reranking. (b) Graph RAG: a graph built from document structure (headings, cross-references) and extracted entities (components, materials, parameters), with retrieval by entity linking and traversal, following [2, 3]. (c) Hybrid: graph traversal collects candidate sections, vector search ranks passages within them. The generator is fixed across systems: one GPT-4-class API model (for example via Azure OpenAI) and one open-weight model to check that findings hold across model sizes. For RQ3, the extracted graph is manually corrected for a subset of 20 to 30 documents.
- **Evaluation:** Retrieval recall of the gold evidence chain; answer correctness via an LLM-as-judge validated against human ratings on a subset; faithfulness [5]; per-hop breakdown of all metrics; indexing time, storage size, and per-query tokens and latency (RQ4); paired significance tests between systems per stratum; an error analysis linking graph defects to answer failures (RQ3).

Expected Contribution

1. **For research:** A controlled, hop-stratified comparison of graph-based and vector retrieval on technical documents, plus a measurement of how extraction errors limit graph retrieval, which prior studies do not isolate.
2. **For practice:** A quality and cost basis for deciding whether building a document graph is justified for a given mix of questions, applicable to maintenance, service, and engineering documentation in any organization.
3. **Open deliverables:** The hop-stratified benchmark with evidence chains, the graph extraction pipeline, and the evaluation code. The corpus is public, so the study can be reproduced.

Proposed Timeline (6 months)

Period	Planned Work
Month 1	Literature review; corpus assembly.
Month 2	Hop-stratified QA set with human verification; vector RAG baseline.
Month 3	Graph extraction; graph RAG and hybrid variants.
Month 4	Manual graph correction for the subset; full experimental runs (RQ1, RQ2).
Month 5	Error propagation analysis (RQ3); cost accounting (RQ4).
Month 6	Thesis writing; code and benchmark release; final presentation.

Preliminary References

- [1] Y. Tang and Y. Yang. *MultiHop-RAG: Benchmarking Retrieval-Augmented Generation for Multi-Hop Queries*. arXiv:2401.15391, 2024.
- [2] *Document GraphRAG: Knowledge Graph Enhanced Retrieval Augmented Generation for Document Question Answering Within the Manufacturing Domain*. Electronics, 14(11):2102, 2025.
- [3] D. Edge et al. *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*. arXiv:2404.16130, 2024.
- [4] P. Lewis et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS, 2020.
- [5] S. Es et al. *RAGAS: Automated Evaluation of Retrieval Augmented Generation*. EACL, 2024.

Student Signature:

Supervisor Signature:
